Object Oriented Programming (CSC166)
Lab Sheet 8

1. Write a program that defines a class "Student" with data members name, roll number, and marks. Write the details of five students to a text file named "students.txt" using formatted output, then read back and display the contents of the file on the screen.

   Hint: [ofstream outFile("students.txt") → open file in output mode to write records
   ifstream inFile("students.txt") → open the same file in input mode to read records
   getData() → member function to input name, roll number and marks for a student
   putData() → member function to display a student's data
   Use outFile << name << roll << marks; to write each student's data on a separate line
   Use inFile >> name >> roll >> marks; inside a while (!inFile.eof()) loop to read until end of file
   Close both files using outFile.close(); and inFile.close(); after use]

2. Write a program that defines a class "Employee" with data members empId, name, and salary. Append new employee records to a binary file "employee.dat" using write(), and display all records currently stored in the file using read().

   Hint: [ofstream outFile("employee.dat", ios::binary | ios::app) → open file in binary and append mode so existing records are preserved
   outFile.write((char) &emp, sizeof(emp)); → write an Employee object as raw bytes*
   ifstream inFile("employee.dat", ios::binary) → open file in binary mode for reading
   inFile.read((char) &emp, sizeof(emp)); → read one Employee object at a time inside a loop*
   Use inFile.read(...) as the loop condition (e.g. while (inFile.read((char)&emp, sizeof(emp)))) so the loop stops automatically at end of file*
   displayEmployee() → prints empId, name, and salary of an Employee object]

3. Write a program that stores 5 integers in a binary file "numbers.dat". Using random access with seekg() and seekp(), allow the user to directly read or modify the value at a given record position (0 to 4) without reading the other records, and also detect and report an invalid position using fail().

   Hint: [fstream file("numbers.dat", ios::in | ios::out | ios::binary | ios::trunc) → open one stream for both reading and writing in binary mode
   file.write((char) &num, sizeof(int)); → write each of the 5 integers sequentially when creating the file*
   *file.seekp(pos * sizeof(int), ios::beg); → move the write pointer directly to the record at index pos before modifying a value*
   *file.seekg(pos * sizeof(int), ios::beg); → move the read pointer directly to the record at index pos before reading a value*
   readValue(int pos) → seeks to pos and reads a single integer using file.read((char) &num, sizeof(int));*
   modifyValue(int pos, int newVal) → seeks to pos and overwrites the integer using file.write((char) &newVal, sizeof(int));*
   After a seek and read/write, check file.fail() (or !file) to detect and report an invalid position such as pos > 4
   Call file.clear(); to reset the error state before reusing the stream after a failed operation]

4. Write a program that defines a class "Book" with data members title, author, and price. Overload the insertion (<<) and extraction (>>) operators for the Book class so that objects can be written to and read from a text file "books.txt" using the same syntax as built-in types, then display all books stored in the file.

   Hint: [friend ostream& operator<<(ostream &out, const Book &b) → overloaded insertion operator that writes title, author and price to the given stream
   friend istream& operator>>(istream &in, Book &b) → overloaded extraction operator that reads title, author and price from the given stream
   ofstream outFile("books.txt") → open the file in output mode, then use outFile << bookObj; to write a Book using the overloaded operator
   ifstream inFile("books.txt") → open the file in input mode, then use inFile >> bookObj; to read a Book using the overloaded operator
   Use a while (inFile >> bookObj) loop to read and display every Book record until end of file
   Return the stream reference (return out; / return in;) from each overloaded operator so operations like cout << b1 << b2; can be chained]